

OS AAT Report:

Given by:

1. Parv Jain (1WA24CS205)
2. Parth Rungta (1WA24CS203)
3. Rishuraj Singh (1WA24CS232)
4. S Manashrest (1WA24CS245)

Submitted to:

**Dr. Seema Patil**

Program 1: **Sequential**

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int disk[50] = {0}; // Initialize 50 disk blocks to 0 (free)
    int startBlock, len, i, j, flag;
    char choice;

    printf("--- Sequential File Allocation ---\n");

    do {
        flag = 0;
        printf("\nEnter starting block (0-49): ");
        scanf("%d", &startBlock);
        printf("Enter length of file: ");
        scanf("%d", &len);

        // Check if blocks are within bounds
        if (startBlock + len > 50) {
            printf("Error: File exceeds disk space.\n");
        } else {
            // Check if contiguous blocks are free
            for (i = startBlock; i < startBlock + len; i++) {
                if (disk[i] == 1) {
                    flag = 1; // Block already allocated
                    break;
                }
            }
        }
    }
}
```

```

        if (flag == 1) {
            printf("Error: Contiguous blocks are not free.\n");
        } else {
            // Allocate the blocks
            for (j = startBlock; j < startBlock + len; j++) {
                disk[j] = 1;
            }
            printf("File allocated successfully from block %d to %d.\n", startBlock, startBlock + len - 1);
        }
    }

    printf("Do you want to allocate another file? (y/n): ");
    scanf(" %c", &choice);
} while (choice == 'y' || choice == 'Y');

return 0;
}

```

Output:

```

Sequential File Allocation
Enter starting block: 5
Enter length of file: 4

Allocated Blocks are:
5 6 7 8
PS D:\tmp\File_Allocation>

```

Program 2: **Indexed**

```

#include <stdio.h>

int main() {
    int disk[50] = {0}; // 0 means free, 1 means allocated
    int indexBlock, n, i, flag;
    int blocks[50];
    char choice;

    printf("--- Indexed File Allocation ---\n");
}

```

```

do {
    printf("\nEnter index block (0-49): ");
    scanf("%d", &indexBlock);

    if (disk[indexBlock] == 1) {
        printf("Error: Index block %d is already allocated.\n", indexBlock);
    } else {
        printf("Enter number of blocks for the file: ");
        scanf("%d", &n);

        printf("Enter the %d block numbers: ", n);
        flag = 0;
        for (i = 0; i < n; i++) {
            scanf("%d", &blocks[i]);
            if (disk[blocks[i]] == 1) {
                flag = 1; // One of the requested blocks is already full
            }
        }

        if (flag == 1) {
            printf("Error: One or more blocks are already allocated. Allocation failed.\n");
        } else {
            // Allocate index block and data blocks
            disk[indexBlock] = 1;
            for (i = 0; i < n; i++) {
                disk[blocks[i]] = 1;
            }
            printf("File allocated successfully.\n");
            printf("Index Block: %d\n", indexBlock);
            printf("Allocated Data Blocks: ");
            for (i = 0; i < n; i++) {
                printf("%d ", blocks[i]);
            }
            printf("\n");
        }
    }

    printf("Do you want to allocate another file? (y/n): ");
    scanf(" %c", &choice);
} while (choice == 'y' || choice == 'Y');

```

```
return 0;  
}
```

Output:

```
Indexed File Allocation  
Enter index block: 3  
Enter number of file blocks: 3  
Enter block numbers:  
7 15 20  
  
Index Block: 3  
Allocated Blocks are:  
7 15 20  
PS D:\tmp\File_Allocation> █
```

Program 3: **Linked**

```
#include <stdio.h>  
#include <stdlib.h>  
  
// Structure to represent a disk block  
struct Block {  
    int allocated;  
    int next;  
};  
  
int main() {  
    struct Block disk[50];  
    int startBlock, n, i, currentBlock, nextBlock;  
    char choice;  
  
    // Initialize disk  
    for (i = 0; i < 50; i++) {  
        disk[i].allocated = 0;  
        disk[i].next = -1; // -1 indicates end of chain  
    }  
  
    printf("--- Linked File Allocation ---\n");  
  
    do {  
        printf("\nEnter starting block (0-49): ");  
        scanf("%d", &startBlock);
```

```

if (disk[startBlock].allocated == 1) {
    printf("Error: Starting block is already allocated.\n");
} else {
    printf("Enter number of blocks required: ");
    scanf("%d", &n);

    disk[startBlock].allocated = 1;
    currentBlock = startBlock;

    // If only 1 block is needed
    if(n == 1) {
        printf("File allocated successfully. Chain: %d -> NULL\n", startBlock);
    }
    else {
        printf("Enter the remaining %d blocks to form the chain:\n", n - 1);
        int success = 1;

        for (i = 1; i < n; i++) {
            printf("Next block after %d: ", currentBlock);
            scanf("%d", &nextBlock);

            if (disk[nextBlock].allocated == 1) {
                printf("Error: Block %d is already allocated! Aborting chain.\n", nextBlock);
                success = 0;
                break;
            } else {
                disk[nextBlock].allocated = 1;
                disk[currentBlock].next = nextBlock;
                currentBlock = nextBlock;
            }
        }
    }

    if (success) {
        printf("File allocated successfully. Chain: ");
        int temp = startBlock;
        while (temp != -1) {
            printf("%d -> ", temp);
            temp = disk[temp].next;
        }
        printf("NULL\n");
    }
}

```

```

    }
}

printf("Do you want to allocate another file? (y/n): ");
scanf(" %c", &choice);
} while (choice == 'y' || choice == 'Y');

return 0;
}

```

Output:

```

Linked File Allocation
Enter number of blocks: 4
Enter block numbers:
7 67 21 91

Linked Allocation:
7 -> 67 -> 21 -> 91 -> NULL
PS D:\tmp\File_Allocation>

```

#### Program 4: **Single Level Directory**

```

#include <stdio.h>
#include <string.h>

int main() {

    int n, i, j, duplicate;
    char files[20][30];
    printf("Single Level Directory\n");

    printf("Enter number of files: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {

        duplicate = 0;

        printf("Enter file name %d: ", i + 1);
        scanf("%s", files[i]);
    }
}

```

```

// Check duplicate filenames
for(j = 0; j < i; j++) {

    if(strcmp(files[i], files[j]) == 0) {

        duplicate = 1;
        break;
    }
}

if(duplicate) {

    printf("File name already exists!\n");
    i--;
}
}

printf("\nFiles in directory:\n");

for(i = 0; i < n; i++) {
    printf("%s\n", files[i]);
}

return 0;
}

```

Output:

```

Single Level Directory
Enter number of files: 2
Enter file name 1: text.txt
Enter file name 2: heap.c

Files in directory:
text.txt
heap.c
PS D:\tmp\File_Organisation>

```

## Program 5: Two Level Directory

```

#include <stdio.h>
#include <string.h>

```

```
int main() {

    int users, i, j, k;
    int fileCount[20];

    char user[20][30];
    char files[20][20][30];
    printf("Two Level Directory\n");

    printf("Enter number of users: ");
    scanf("%d", &users);

    for(i = 0; i < users; i++) {

        printf("\nEnter username: ");
        scanf("%s", user[i]);

        printf("Enter number of files for %s: ", user[i]);
        scanf("%d", &fileCount[i]);

        for(j = 0; j < fileCount[i]; j++) {

            int duplicate = 0;

            printf("Enter file %d: ", j + 1);
            scanf("%s", files[i][j]);

            // Check duplicate within same user
            for(k = 0; k < j; k++) {

                if(strcmp(files[i][j], files[i][k]) == 0) {

                    duplicate = 1;
                    break;
                }
            }

            if(duplicate) {

                printf("Duplicate file in same directory not allowed!\n");
```

```

        j--;
    }
}

printf("\nDirectory Structure:\n");

for(i = 0; i < users; i++) {

    printf("\nUser: %s\n", user[i]);

    for(j = 0; j < fileCount[i]; j++) {
        printf("%s\n", files[i][j]);
    }
}

return 0;
}

```

Output:

```

Two Level Directory
Enter number of users: 2

Enter username: Person1
Enter number of files for Person1: 2
Enter file 1: Accounts.txt
Enter file 2: Game.png

Enter username: Person2
Enter number of files for Person2: 2
Enter file 1: Accounts.txt
Enter file 2: Image.jpg

Directory Structure:

User: Person1
Accounts.txt
Game.png

User: Person2
Accounts.txt
Image.jpg
PS D:\tmp\File_Organisation>

```

## Program 6: Hierarchical Level Directory

```
#include <stdio.h>
```

```
#include <string.h>

int main() {

    int mainDir, i, j, k, l;
    int subCount[20];
    int fileCount[20][20];

    char mainDirectory[20][30];
    char subDirectory[20][20][30];
    char files[20][20][20][30];
    printf("Hierarchical Directory\n");

    printf("Enter number of main directories: ");
    scanf("%d", &mainDir);

    for(i = 0; i < mainDir; i++) {

        printf("\nEnter main directory name: ");
        scanf("%s", mainDirectory[i]);

        printf("Enter number of subdirectories in %s: ", mainDirectory[i]);
        scanf("%d", &subCount[i]);

        for(j = 0; j < subCount[i]; j++) {

            printf("Enter subdirectory name: ");
            scanf("%s", subDirectory[i][j]);

            printf("Enter number of files in %s: ", subDirectory[i][j]);
            scanf("%d", &fileCount[i][j]);

            for(k = 0; k < fileCount[i][j]; k++) {

                int duplicate = 0;

                printf("Enter file %d: ", k + 1);
                scanf("%s", files[i][j][k]);

                // Check duplicate inside same subdirectory
                for(l = 0; l < k; l++) {
```

```

        if(strcmp(files[i][j][k], files[i][j][l]) == 0) {

            duplicate = 1;
            break;
        }
    }

    if(duplicate) {

        printf("Duplicate file in same folder not allowed!\n");
        k--;
    }
}
}

printf("\nHierarchical Directory Structure:\n");

for(i = 0; i < mainDir; i++) {

    printf("\n%s\n", mainDirectory[i]);

    for(j = 0; j < subCount[i]; j++) {

        printf("  %s\n", subDirectory[i][j]);

        for(k = 0; k < fileCount[i][j]; k++) {

            printf("    %s\n", files[i][j][k]);
        }
    }
}

return 0;
}

```

Output:

```

Hierarchical Directory
Enter number of main directories: 2

Enter main directory name: Document
Enter number of subdirectories in Document: 2
Enter subdirectory name: College
Enter number of files in College: 1
Enter file 1: Assignment.docx
Enter subdirectory name: Notes
Enter number of files in Notes: 1
Enter file 1: OS.txt

Enter main directory name: Picture
Enter number of subdirectories in Picture: 1
Enter subdirectory name: photo.jpg
Enter number of files in photo.jpg: 0

Hierarchical Directory Structure:

Document
  College
    Assignment.docx
  Notes
    OS.txt

Picture
  photo.jpg
PS D:\tmp\File_Organisation>

```

## Program 7: FCFS

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, head, i;
    int queue[20];
    int totalSeek = 0;
    printf("FCFS Disk Scheduling\n");

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &queue[i]);
    }
}

```

```

printf("Enter initial head position: ");
scanf("%d", &head);

printf("\nSeek Sequence:\n");
printf("%d ", head);

for(i = 0; i < n; i++) {
    totalSeek += abs(queue[i] - head);
    head = queue[i];
    printf("-> %d ", head);
}

printf("\n\nTotal Seek Time = %d\n", totalSeek);

return 0;
}

```

Output:

```

FCFS Disk Scheduling
Enter number of requests: 5
Enter request queue:
98 183 37 122 14
Enter initial head position: 53

Seek Sequence:
53 -> 98 -> 183 -> 37 -> 122 -> 14

Total Seek Time = 469
PS D:\tmp\Disk_Scheduling>

```

Program 8: **SCAN**

```

#include <stdio.h>
#include <stdlib.h>

void sort(int arr[], int n) {
    int i, j, temp;
    for(i = 0; i < n - 1; i++) {
        for(j = 0; j < n - i - 1; j++) {
            if(arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
            }
        }
    }
}

```

```

        arr[j + 1] = temp;
    }
}
}

int main() {
    int queue[20], n, head, diskSize;
    int direction;
    int i, j;
    int totalSeek = 0;
    printf("SCAN Disk Scheduling\n");
    printf("Enter number of requests: ");
    scanf("%d", &n);
    printf("Enter request queue:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &queue[i]);
    }
    printf("Enter initial head position: ");
    scanf("%d", &head);
    printf("Enter disk size: ");
    scanf("%d", &diskSize);
    printf("Enter direction (0 for left, 1 for right): ");
    scanf("%d", &direction);
    sort(queue, n);
    printf("\nSeek Sequence:\n");
    printf("%d -> ", head);
    for(i = 0; i < n; i++) {
        if(queue[i] >= head)
            break;
    }

    // RIGHT direction
    if(direction == 1) {
        for(j = i; j < n; j++) {
            printf("%d -> ", queue[j]);
            totalSeek += abs(queue[j] - head);
            head = queue[j];
        }
        printf("%d -> ", diskSize - 1);
        totalSeek += abs((diskSize - 1) - head);
    }
}

```

```

    head = diskSize - 1;
    for(j = i - 1; j >= 0; j--) {
        printf("%d -> ", queue[j]);
        totalSeek += abs(queue[j] - head);
        head = queue[j];
    }
}

// LEFT direction
else {
    for(j = i - 1; j >= 0; j--) {
        printf("%d -> ", queue[j]);
        totalSeek += abs(queue[j] - head);
        head = queue[j];
    }
    printf("0 -> ");
    totalSeek += abs(head - 0);
    head = 0;
    for(j = i; j < n; j++) {
        printf("%d -> ", queue[j]);
        totalSeek += abs(queue[j] - head);
        head = queue[j];
    }
}

printf("\n\nTotal Seek Time = %d\n", totalSeek);
return 0;
}

```

Output:

```

SCAN Disk Scheduling
Enter number of requests: 5
Enter request queue:
98 183 37 122 14
Enter initial head position: 53
Enter disk size: 200
Enter direction (0 for left, 1 for right): 1

Seek Sequence:
53 -> 98 -> 122 -> 183 -> 199 -> 37 -> 14 ->

Total Seek Time = 331

```

## Program 9: SSTF

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int queue[20], visited[20] = {0};
    int n, head, i, totalSeek = 0;
    printf("SSTF Disk Scheduling\n");

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &queue[i]);
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    printf("\nSeek Sequence:\n");

    for(i = 0; i < n; i++) {

        int min = 9999;
        int index = -1;
        int j;

        for(j = 0; j < n; j++) {

            if(!visited[j]) {

                int distance = abs(queue[j] - head);

                if(distance < min) {
                    min = distance;
                    index = j;
                }
            }
        }
    }
}
```

```

        visited[index] = 1;
        totalSeek += min;
        head = queue[index];

        printf("%d -> ", head);
    }

    printf("\n\nTotal Seek Time = %d\n", totalSeek);

    return 0;
}

```

Output:

```

SSTF Disk Scheduling
Enter number of requests: 5
Enter request queue:
98 183 37 122 14
Enter initial head position: 53

Seek Sequence:
37 -> 14 -> 98 -> 122 -> 183 ->

Total Seek Time = 208
> PS D:\tmp\Disk_Scheduling> █

```

Program 10: **C-LOOK**

```

#include <stdio.h>
#include <stdlib.h>

void sort(int arr[], int n) {

    int i, j, temp;

    for(i = 0; i < n - 1; i++) {

        for(j = 0; j < n - i - 1; j++) {

            if(arr[j] > arr[j + 1]) {

                temp = arr[j];

```

```

        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}
}

int main() {

    int queue[20], n, head;
    int i, j;
    int totalSeek = 0;
    printf("C-LOOK Disk Scheduling\n");

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue:\n");

    for(i = 0; i < n; i++) {
        scanf("%d", &queue[i]);
    }

    printf("Enter initial head position: ");
    scanf("%d", &head);

    sort(queue, n);

    printf("\nSeek Sequence:\n");
    printf("%d -> ", head);

    // Find first request greater than head
    for(i = 0; i < n; i++) {

        if(queue[i] >= head)
            break;
    }

    // Service requests to the right
    for(j = i; j < n; j++) {

```

```

        printf("%d -> ", queue[j]);

        totalSeek += abs(queue[j] - head);

        head = queue[j];
    }

    // Jump to first request (NOT counted)
    if(i > 0) {

        printf("%d -> ", queue[0]);

        head = queue[0];
    }

    // Continue servicing remaining requests
    for(j = 1; j < i; j++) {

        printf("%d -> ", queue[j]);

        totalSeek += abs(queue[j] - head);

        head = queue[j];
    }

    printf("\n\nTotal Seek Time = %d\n", totalSeek);

    return 0;
}

```

Output:

```

C-LOOK Disk Scheduling
Enter number of requests: 5
Enter request queue:
98 183 37 122 14
Enter initial head position: 53

Seek Sequence:
53 -> 98 -> 122 -> 183 -> 14 -> 37 ->

Total Seek Time = 153
PS D:\tmp\Disk Scheduling>

```